Codex Implementation Brief: Native iOS Portuguese-to-English Live Interpreter

1. Objective

Build a native SwiftUI iPhone application that acts as an always-ready, one-way live interpreter:

- Input: spoken Portuguese
- Output: English captions and optionally translated English audio
- Primary speaker: a 95-year-old woman from Casegas, Portugal
- Speech characteristics:
- European Portuguese
- Older regional vocabulary and pronunciation
- Sometimes quiet, unclear, fragmented, or incomplete
- Occasional Portuguese mixed with English
- Usage pattern:
- The user opens the app during an existing conversation.
- The app begins listening immediately.
- The user does not have to enter a prompt, select Portuguese, or press a start button after initial setup.
- The app should remain visually subtle and should not interrupt the conversation.

Use OpenAI's dedicated `gpt-realtime-translate` model. Do not build this as a prompted general-purpose voice assistant.

Realtime Translation uses a dedicated continuous translation endpoint, automatically translates incoming audio, produces translated speech and transcript deltas, and does not use the normal assistant-turn or `response.create` lifecycle.

---

2. Core Product Requirements

2.1 Immediate operation

After microphone permission has been granted once:

1. User opens the application.
2. Application configures the audio session.
3. Application obtains a short-lived OpenAI client secret.
4. Application establishes a Realtime Translation connection.
5. Application begins sending microphone audio continuously.
6. English captions begin appearing when Portuguese is detected.
7. English audio plays only when the selected output mode permits it.

No start button should be required during normal use.

A visible Pause/Resume control must still be available.

2.2 Output modes

Implement a segmented control with three modes:

```
enum TranslationOutputMode: String, CaseIterable, Codable {
    case automatic
    case listen
    case read
}
```

Automatic

This is the default.

- Private audio route detected:
- Play translated English audio.
- Display English captions.
- No private audio route:
- Mute translated English audio locally.
- Display English captions only.
- If headphones disconnect:
- Mute translated audio immediately.
- If headphones connect:
- Enable translated audio automatically.

Listen

- Play translated English audio through the active system output.
- Continue displaying captions.
- If the active route is the iPhone speaker, show a small nonblocking warning:
- “Speaker audio may be heard by others.”
- Do not repeatedly show the warning after it has been acknowledged.

Read

- Never play translated English audio.
- Display live English captions.
- Continue receiving the remote audio track so the connection does not need to restart when modes change.
- Mode changes must be instantaneous and local.

The OpenAI session provides translated audio and transcript events concurrently, so output mode changes should mute or unmute the remote audio track rather than recreate the translation session.

3. Architecture Decision

Build the MVP as:

```
SwiftUI iOS application
    │
    ├── AVAudioSession configuration
    ├── Native WebRTC peer connection
    ├── OpenAI Realtime Translation session
    ├── English caption rendering
    └── Local translated-audio muting/routing

Small token service
    │
    └── Creates short-lived OpenAI translation client secrets
```

Why native iOS

A native application provides better control over:

- Microphone permission
- Audio-session category and mode
- Preferred microphone input
- Route changes
- Bluetooth and wired headphone behavior
- Application lifecycle
- Audio interruptions
- Keeping the screen awake
- Automatic startup behavior
- Diagnostics for the actual microphone route

Why WebRTC for the MVP

Use WebRTC as the primary transport because it:

- Sends microphone audio as a media track.
- Receives translated speech as a remote media track.
- Handles resampling, jitter, packet loss, encoding, and audio playback.
- Uses substantially less bandwidth than manually streaming uncompressed PCM.
- Allows captions to arrive independently through the `oai-events` data channel.
- Matches OpenAI's documented client-side translation flow.

OpenAI recommends WebRTC when client software captures or plays audio, while raw WebSockets require manually sending 24 kHz PCM16 and manually playing returned PCM chunks.

Native WebRTC qualification

OpenAI's published example uses browser WebRTC APIs. This project will implement the equivalent SDP offer/answer flow using a native iOS WebRTC library.

Before building the full interface, complete a technical spike proving that the selected native WebRTC package can:

1. Create an audio peer connection.
2. Create the `oai-events` data channel.
3. Produce a valid SDP offer.
4. POST that offer to the OpenAI translation calls endpoint.
5. Apply the returned SDP answer.
6. Send live microphone audio.
7. Receive a translated remote audio track.
8. Receive `session.output_transcript.delta` events.

Use a maintained WebRTC XCFramework or Swift Package compatible with the installed Xcode version. Pin the exact dependency version in the repository.

Do not embed the permanent OpenAI API key in the iOS application.

---

4. OpenAI Translation Session

4.1 Model

Use:

```
gpt-realtime-translate
```

Target language:

```
en
```

Enable source-language transcription:

```
gpt-realtime-whisper
```

The translation model automatically detects the source language. The client only needs to configure the output language. Source transcription is auxiliary; translation runs directly from the original audio, not from the transcription.

4.2 Initial session configuration

For the iPhone built-in microphone, default to `far_field` noise reduction because the phone will commonly sit on a table or be positioned between speakers.

```
{
  "session": {
    "model": "gpt-realtime-translate",
    "audio": {
      "input": {
        "transcription": {
          "model": "gpt-realtime-whisper"
        },
        "noise_reduction": {
          "type": "far_field"
        }
      },
      "output": {
        "language": "en"
      }
    }
  },
  "expires_after": {
    "anchor": "created_at",
    "seconds": 600
  }
}
```

OpenAI defines:

- `near_field` for close-talking inputs such as headset microphones.
- `far_field` for room, laptop, or conference-style microphone placement.

Dynamically select the profile:

```
enum OpenAINoiseReductionProfile: String {
    case nearField = "near_field"
    case farField = "far_field"
}
```

Recommended mapping:

```
Built-in iPhone microphone          → far_field
Wired headset microphone            → near_field
```

```
Bluetooth HFP microphone            → near_field
Unknown or external room microphone → far_field
```

When the active input route changes, send a `session.update` event if needed rather than recreating the connection.

4.3 WebRTC connection flow

Implement the following sequence:

1. Request a translation client secret from the token service.
2. Configure and activate `AVAudioSession`.
3. Create the WebRTC peer-connection factory.
4. Create an audio source and local microphone audio track.
5. Add the audio track to the peer connection.
6. Create a data channel named exactly:

```
oai-events
```

1. Set the remote-track handler.
2. Create an SDP offer.
3. Set it as the local description.
4. POST the SDP to:

```
https://api.openai.com/v1/realtime/translations/calls
```

Headers:

```
Authorization: Bearer <short-lived-client-secret>
Content-Type: application/sdp
```

1. Read the SDP answer from the HTTP response.
2. Set the answer as the remote description.
3. Mark the state as listening once the peer connection is connected and the session-created event has arrived.

This mirrors OpenAI's documented browser translation-call flow.

4.4 Events to support

Decode at least:

```
session.created
session.updated
session.closed
session.input_transcript.delta
session.output_transcript.delta
error
```

Event model:

```swift
struct TranslationServerEvent: Decodable {
    let type: String
    let eventId: String?
    let delta: String?
    let elapsedMs: Int?
    let error: TranslationAPIError?

    enum CodingKeys: String, CodingKey {
        case type
        case eventId = "event_id"
        case delta
        case elapsedMs = "elapsed_ms"
        case error
    }
}
```

Transcript deltas are append-only. Append the exact delta text and do not automatically insert spaces between events. Multiple events may share the same `elapsed_ms`; treat it as alignment metadata, not as a unique event identifier.

---

5. Token Service

Create a minimal TypeScript service with one route:

```
POST /api/translation-token
```

The backend must hold:

```
OPENAI_API_KEY
```

The iOS app must never receive or store the permanent API key.

Request

```
{
  "noiseReduction": "far_field",
  "deviceId": "stable-random-installation-id"
}
```

Only accept whitelisted values:

```
near_field
far_field
```

Hard-code the model and English output language server-side. Do not allow the client to specify arbitrary models or API payloads.

Response

```
{
  "clientSecret": "ek_...",
  "expiresAt": 1780000000
}
```

Example server behavior

```
app.post("/api/translation-token", async (req, res) => {
  const requestedNoiseReduction = req.body?.noiseReduction;

  const noiseReduction =
    requestedNoiseReduction === "near_field"
      ? "near_field"
      : "far_field";

  const deviceId = String(req.body?.deviceId ?? "unknown");
  const safetyIdentifier = hashDeviceId(deviceId);

  const response = await fetch(
    "https://api.openai.com/v1/realtime/translations/client_secrets",
    {
      method: "POST",
      headers: {
        Authorization: `Bearer ${process.env.OPENAI_API_KEY}`,
        "Content-Type": "application/json",
        "OpenAI-Safety-Identifier": safetyIdentifier
```

```
      },
      body: JSON.stringify({
        session: {
          model: "gpt-realtime-translate",
          audio: {
            input: {
              transcription: {
                model: "gpt-realtime-whisper"
              },
              noise_reduction: {
                type: noiseReduction
              }
            },
            output: {
              language: "en"
            }
          }
        },
        expires_after: {
          anchor: "created_at",
          seconds: 600
        }
      })
    }
  );

  const body = await response.json();

  if (!response.ok) {
    res.status(response.status).json({
      error: "translation_token_failed",
      details: body
    });
    return;
  }

  res.json({
    clientSecret: body.value,
    expiresAt: body.expires_at
  });
});
```

OpenAI's translation-client-secret endpoint is specifically intended to create short-lived credentials for translation sessions. A secret may expire for the creation of new sessions while an already-created session continues running.

Add:

- Basic rate limiting
- Request timeouts
- Structured error logging
- An `.env.example`
- No secrets committed to source control

For this private side-loaded application, full user authentication is optional. At minimum, protect the endpoint with a random application access token stored in the iOS Keychain and validated by the server.

---

6. iOS Project Structure

Use a structure similar to:

```
Interpreter/
├── App/
│   ├── InterpreterApp.swift
│   └── AppEnvironment.swift
├── Models/
│   ├── TranslationOutputMode.swift
│   ├── TranslationConnectionState.swift
│   ├── TranslationServerEvent.swift
│   ├── AudioRouteSnapshot.swift
│   └── TranscriptEntry.swift
├── Services/
│   ├── RealtimeTranslationClient.swift
│   ├── WebRTCPeerConnection.swift
│   ├── TranslationTokenService.swift
│   ├── AudioSessionController.swift
│   ├── AudioRouteMonitor.swift
│   ├── TranscriptAssembler.swift
│   ├── AppSettingsStore.swift
│   └── DiagnosticsRecorder.swift
├── ViewModels/
│   └── InterpreterViewModel.swift
├── Views/
│   ├── InterpreterView.swift
│   ├── CaptionView.swift
│   ├── OutputModePicker.swift
│   ├── ConnectionStatusView.swift
│   ├── SettingsView.swift
│   └── DiagnosticsView.swift
├── Utilities/
```

```
│   ├── Logger.swift
│   └── KeychainStore.swift
└── Resources/
    └── Assets.xcassets

InterpreterTests/
├── TranscriptAssemblerTests.swift
├── OutputModeRoutingTests.swift
├── EventDecodingTests.swift
└── ConnectionStateTests.swift

server/
├── src/
│   ├── index.ts
│   ├── openai.ts
│   └── rateLimit.ts
├── package.json
├── tsconfig.json
└── .env.example
```

Keep WebRTC-specific code behind a protocol so it can be replaced without changing the ViewModel:

```swift
protocol TranslationTransport: AnyObject {
    var delegate: TranslationTransportDelegate? { get set }

    func connect(configuration: TranslationSessionConfiguration) async throws
    func disconnect()
    func setRemoteAudioEnabled(_ enabled: Bool)
    func sendSessionUpdate(_ update: TranslationSessionUpdate) throws
}
```

---

7. Application State Machine

Implement an explicit state machine:

```swift
enum TranslationConnectionState: Equatable {
    case idle
    case requestingMicrophonePermission
    case configuringAudio
    case requestingToken
    case connecting
    case listening
    case paused
```

```
    case reconnecting(attempt: Int)
    case failed(message: String)
    case stopped
}
```

Expected normal launch:

```
idle
→ requestingMicrophonePermission, first launch only
→ configuringAudio
→ requestingToken
→ connecting
→ listening
```

On recoverable network failure:

```
listening
→ reconnecting
→ requestingToken
→ connecting
→ listening
```

On pause:

```
listening
→ paused
```

Pause should disable or remove the outgoing microphone track without destroying caption history.

After a long pause, it is acceptable to recreate the full connection when resuming.

---

8. Audio Session and Microphone Strategy

Audio quality is the highest-priority engineering concern.

8.1 Default audio-session configuration

Configure `AVAudioSession` before creating the WebRTC audio track.

Use:

```
Category: playAndRecord
Initial mode: videoChat
Bluetooth HFP input: permitted
Bluetooth A2DP output: permitted
```

Use the current SDK's supported Bluetooth option names. New SDKs may expose `allowBluetoothHFP`; older SDKs may use `allowBluetooth`.

Do not use `mixWithOthers` by default. Other device audio should not be mixed into the microphone/playback session unnecessarily.

Apple documents that the selected audio-session mode affects routing and the digital signal processing applied to microphone input. It also provides APIs for requesting a preferred input, selecting microphone data sources, and selecting supported polar patterns.

8.2 Preferred microphone

For the expected use case, prefer the iPhone's built-in microphone even when headphones are connected.

Reason:

- The phone can be positioned close to the grandmother.
- AirPods or another headset microphone may be near the user instead of the speaker.
- A microphone near the wrong person would substantially reduce recognition quality.

Process:

1. Activate the audio session.
2. Inspect `availableInputs`.
3. Find `.builtInMic`.
4. Request it with `setPreferredInput`.
5. Inspect `currentRoute.inputs` afterward.
6. Never assume the request succeeded.

Apple describes `setPreferredInput` as a request; the effective result should be confirmed through `currentRoute`.

Expose the active input source in the diagnostics screen:

```
Input: Built-in microphone
Output: AirPods
Noise profile: Far field
Audio mode: Video chat
```

8.3 AirPods and Bluetooth caveat

The combination:

```
Built-in iPhone microphone input
+
Bluetooth headphone output
```

may vary based on the device, Bluetooth profile, iOS version, and WebRTC audio-session behavior.

Do not assume it works until verified on the actual iPhone and headphones.

If connecting AirPods forces the AirPods microphone:

- Clearly display this in Diagnostics.
- Allow the user to switch to Read mode.
- Prefer the phone microphone and captions over translated audio if that gives better source capture.
- Do not silently sacrifice recognition quality merely to preserve headphone playback.

8.4 Audio-session profile testing

Implement two profiles behind a debug setting:

```
enum AudioProcessingProfile: String, CaseIterable {
    case videoChat
    case voiceChat
}
```

Default:

```
videoChat
```

Test both profiles using actual recordings of the target speaker.

Do not assume stronger processing is always better. Noise suppression can help room noise, but overly aggressive processing may damage quiet or breathy speech.

Persist the best-performing profile after testing.

8.5 Microphone data sources and polar patterns

Do not hard-code a front, back, top, or bottom microphone in the first implementation.

Different iPhone models expose different microphone data sources. Instead:

1. Enumerate supported built-in microphone data sources in the diagnostics screen.
2. Display location and orientation.
3. Display supported polar patterns.
4. Allow an advanced manual selection.
5. Record which selection performs best on the user's actual phone.
6. Persist that selection.

Only apply a directional pattern if:

- The device supports it.
- Testing demonstrates a meaningful improvement.
- It does not cut off speech when the phone shifts slightly.

Apple documents that built-in microphone data sources and polar-pattern support vary by device.

8.6 Continuous capture

Do not implement a local volume threshold that starts and stops microphone transmission.

Do not gate audio based on local VAD.

The microphone track should remain active while the application is listening, including during pauses and silence. This prevents quiet syllables and the beginnings of sentences from being clipped.

OpenAI's translation sessions are continuous and expect audio, including silence between phrases.

8.7 Quiet-speech indicator

Display a small, nonintrusive microphone-quality indicator:

```
Good
Move phone closer
Very quiet
High background noise
```

This indicator must never stop transmission.

If the chosen WebRTC library does not expose microphone levels cleanly, do not add a competing parallel microphone capture pipeline merely for visualization. Start with route diagnostics and add a custom WebRTC audio-device integration only if needed.

Do not apply arbitrary digital gain in the MVP. Automatic gain control may already be applied by the WebRTC and iOS audio paths. Additional gain can amplify room noise or cause clipping.

---

9. Audio Route Monitoring

Observe:

```
AVAudioSession.routeChangeNotification
AVAudioSession.interruptionNotification
AVAudioSession.mediaServicesWereResetNotification
```

Apple provides route-change notifications for added and removed audio inputs and outputs. Re-read `currentRoute` after every change.

Represent the current route:

```
struct AudioRouteSnapshot: Equatable {
    let inputPortType: AVAudioSession.Port
    let inputName: String
    let outputPortTypes: [AVAudioSession.Port]
    let outputNames: [String]
    let hasPrivateOutput: Bool
}
```

Private output should include likely personal-listening routes:

```
Wired headphones
Bluetooth A2DP
Bluetooth HFP
Bluetooth LE
Hearing aid
```

Be conservative. A Bluetooth route type may also represent a speaker or vehicle, so provide a setting allowing the user to mark a route as private or public.

When a route changes:

1. Refresh the route snapshot.
2. Re-evaluate the preferred microphone.
3. Re-evaluate the OpenAI noise-reduction profile.
4. Send `session.update` if the profile changed.
5. Recompute whether translated audio should play.
6. Update the diagnostics screen.

---

10. Output Mode Routing Logic

Implement one source of truth:

```
func shouldPlayTranslatedAudio(
    mode: TranslationOutputMode,
    route: AudioRouteSnapshot
) -> Bool {
    switch mode {
    case .automatic:
        return route.hasPrivateOutput

    case .listen:
        return true

    case .read:
        return false
    }
}
```

On every output-mode or route change:

```
transport.setRemoteAudioEnabled(
    shouldPlayTranslatedAudio(
        mode: settings.outputMode,
        route: audioRouteMonitor.currentRoute
    )
)
```

Persist the selected mode with `AppStorage`.

Default:

```
automatic
```

If the app is launched with no private output, translated audio must begin muted.

Never briefly play translated audio through the phone speaker while the route is still being resolved.

---

11. Caption Handling

11.1 Live caption display

The main English caption should:

- Update incrementally.
- Use large Dynamic Type.
- Support multiline text.
- Remain readable at arm's length.
- Avoid jumping excessively as text arrives.
- Keep the latest text visible.
- Preserve recent completed text in a scrollable history.

Maintain:

```
struct TranscriptState {
    var liveEnglishText: String
    var livePortugueseText: String
    var history: [TranscriptEntry]
}
```

Append deltas exactly:

```
func appendEnglishDelta(_ delta: String) {
    liveEnglishText.append(delta)
}
```

Do not do:

```
liveEnglishText += " " + delta
```

because OpenAI transcript deltas may already include their required spacing.

11.2 Segmenting transcript history

The translation API provides streaming deltas rather than relying on normal assistant-turn completion.

For the MVP, create a completed caption segment when any of these occurs:

- No English transcript delta for approximately 1.2 seconds.
- Strong terminal punctuation is received and no further delta follows for approximately 500 milliseconds.

- The live caption exceeds a reasonable length, such as 300 characters.
- The session disconnects or pauses.

Make these timing values configurable constants.

When a segment completes:

1. Copy it to history.
2. Retain it on screen.
3. Clear the active live segment only after the history row has appeared.
4. Never lose the most recently displayed translation.

11.3 Portuguese source transcript

Enable source transcription, but hide it by default.

Add a setting:

```
Show original Portuguese
```

When enabled:

- Display Portuguese below English in smaller, lower-contrast text.
- Label it “Portuguese heard.”
- Treat it as diagnostic guidance, not guaranteed ground truth.

11.4 Caption history

Keep caption history in memory for the current session.

Default behavior:

- Do not write transcripts to disk.
- Clear history when the user explicitly stops the session or closes the app.
- Provide a manual “Copy session text” action.
- Do not automatically upload transcript history anywhere other than the live API stream.

---

12. Main Interface

The default interface should contain only:

```
┌──────────────────────────────────────┐
│ Listening                • Live      │
│                                      │
```

```
│ She said they used to visit the   │
│ convent when they were younger…   │
│                                   │
│ Previous translated sentence      │
│ remains visible below.            │
│                                   │
│ [ Auto ] [ Listen ] [ Read ]      │
│                                   │
│        Pause           Stop       │
└───────────────────────────────────┘
```

Main-screen elements

- Connection status
- Subtle animated listening indicator
- Large English captions
- Recent caption history
- Output-mode picker
- Pause/Resume
- Stop
- Small settings button

Do not include:

- Chat bubbles
- Text prompt field
- Send button
- Assistant avatar
- Language selector on the main screen
- Unnecessary onboarding after the first launch

Status text

Use simple states:

```
Starting…
Connecting…
Listening
Paused
Reconnecting…
Microphone permission required
No network
Unable to connect
```

Haptics

Use one light haptic when:

- The application successfully begins listening.
- The user pauses.
- The user resumes.

Do not play a startup sound.

Screen behavior

While listening:

```
UIApplication.shared.isIdleTimerDisabled = true
```

Restore the normal idle-timer behavior when paused, stopped, or backgrounded.

---

13. Launch and Lifecycle Behavior

First launch

1. Explain in one screen:

2. The app listens through the microphone.

3. Audio is sent to OpenAI for live translation.
4. English can be read or heard through headphones.
5. Request microphone permission.
6. Default to Automatic output mode.
7. Start the session after permission is granted.

Required `Info.plist` text:

```
NSMicrophoneUsageDescription:
This app uses the microphone to translate nearby Portuguese speech into English.
```

Subsequent launches

Start automatically from the root view's task or application-active lifecycle event.

Do not require a tap unless:

- Microphone permission was denied.

- The user previously selected an explicit “Do not auto-start” setting.
- A fatal configuration error exists.

Backgrounding

For the MVP:

- Stop or suspend translation when the application enters the background.
- Resume automatically when it returns to the foreground, unless the user explicitly stopped it.
- Do not enable background microphone capture initially.

This reduces privacy risk and avoids unnecessary API usage.

Interruptions

On phone calls, Siri, or other audio interruptions:

1. Mark the app paused.
2. Disable translated playback.
3. Wait for interruption-ended notification.
4. Reactivate `AVAudioSession`.
5. Verify routes.
6. Reconnect if required.
7. Resume automatically.

---

14. Reconnection Strategy

Reconnect after:

- ICE failure
- Peer-connection failure
- Session closed unexpectedly
- Temporary network loss
- Media-services reset
- Invalid or expired client secret during connection

Use bounded exponential backoff:

```
Attempt 1: immediate
Attempt 2: 1 second
Attempt 3: 2 seconds
Attempt 4: 4 seconds
Attempt 5: 8 seconds
Maximum delay: 10 seconds
```

Fetch a new client secret for each new connection attempt.

After five consecutive failures:

- Stay on the screen.
- Show a clear Retry button.
- Preserve existing captions.
- Do not crash.
- Do not loop indefinitely at high frequency.

When connectivity returns, retry automatically if the user has not pressed Stop.

---

15. Accuracy Strategy for Older European Portuguese

The dedicated translation model currently does not support custom prompts or fixed voice selection. Therefore, it cannot directly be told to prioritize Casegas vocabulary, family names, or older regional expressions.

The MVP should still use the dedicated translation model because it is optimized for continuous interpretation and consumes the original audio directly.

Phase-two optional correction layer

After the live translator works, add an optional text-only correction path.

Inputs:

- Latest Portuguese source-transcript segment
- Latest English translation segment
- Previous one or two segments for context
- User-maintained glossary:
- Family names
- Casegas
- Nearby places
- Religious vocabulary
- Older expressions
- Commonly misheard words

Process:

1. Wait until a caption segment is considered complete.
2. Send the source transcript and first English translation to a server-side text model.
3. Request a corrected English rendering.
4. Replace only the displayed caption.
5. Mark revised captions subtly.
6. Do not delay the initial live caption.

7. Do not replace translated audio in the MVP.

Example correction request:

```
Correct the English translation using the Portuguese source transcript.

Context:
- Speaker uses older European Portuguese.
- Speaker grew up in Casegas, Portugal.
- Preserve uncertainty when the source is unclear.
- Preserve names, numbers, dates, and locations.
- Do not add facts.
- Return only corrected English.

Portuguese heard:
...

Initial English translation:
...
```

This correction path must be optional because the source transcript can itself contain errors.

---

16. Background Voices and Television

The application cannot reliably know which Portuguese speaker is the grandmother from a single mixed microphone stream.

It may translate:

- Portuguese television audio
- Another Portuguese speaker
- Portuguese audio from a nearby device

Do not claim speaker isolation in the MVP.

Mitigation:

- Place the iPhone closer to the intended speaker than to the television.
- Prefer the built-in phone microphone.
- Test supported directional microphone configurations.
- Use `far_field` OpenAI noise reduction for table placement.
- Add a clear audio-input indicator.
- Add an optional “TV noise test” to diagnostics.

English background speech may produce no translated output because English is already the configured output language, but this should not be treated as a strict filtering guarantee. OpenAI notes that same-language input may result in silence.

Future speaker-lock functionality would require speaker enrollment or diarization and is outside MVP scope.

---

17. Diagnostics and Evaluation Mode

Add a hidden or settings-based Diagnostics screen.

Display

- Connection state
- Session duration
- Peer-connection state
- ICE state
- Data-channel state
- Active microphone input
- Active audio output
- Selected audio-session mode
- OpenAI noise-reduction profile
- Selected microphone data source
- Selected polar pattern
- Last API error
- Approximate transcript latency
- Reconnection count

Test controls

- Audio profile:
- Video Chat
- Voice Chat
- OpenAI noise reduction:
- Far Field
- Near Field
- Disabled, diagnostic only
- Built-in microphone data source
- Supported polar pattern
- Show Portuguese transcript
- Save diagnostic log
- Record a short local test clip, explicit action only

Privacy

Diagnostic recording must:

- Require an explicit button press.
- Show a visible recording indicator.
- Save locally only.
- Never upload automatically.
- Be easy to delete.
- Be disabled by default.

Evaluation set

With the speaker's knowledge, create a small test set containing:

- Normal Casegas Portuguese
- Quiet speech
- Fragmented sentences
- Long stories
- Names of family members
- Place names
- Numbers, dates, and ages
- Religious terms
- Portuguese mixed with English
- Television in the background
- Phone 30–60 cm away
- Phone farther away
- Headphones connected and disconnected

For each test, retain:

- Original source audio
- Portuguese source transcript
- English translated transcript
- Translated audio
- Audio profile
- Active microphone route
- Latency timings
- Human-reviewed expected meaning

Evaluate separately:

1. What did the system hear?
2. What English meaning did it produce?
3. What should it have produced?
4. How long did the translation take to arrive?

OpenAI recommends evaluating meaning, names, numbers, tone, skipped content, and latency independently rather than relying only on exact text matching.

---

18. Logging

Use structured logging with categories:

```
app
audio
route
webrtc
openai
transcript
reconnect
diagnostics
```

Production/default logs must not include:

- Raw audio
- Full transcripts
- Client secrets
- Permanent API keys
- Token-service authorization values

Logs may include:

- Event type
- Timestamp
- Connection state
- Error code
- Route type
- Elapsed latency
- Transcript character count

Allow transcript logging only through an explicit diagnostics setting.

---

19. Security Requirements

- Permanent OpenAI API key exists only on the token server.
- Translation client secrets remain in memory only.
- Store the private token-service access credential in Keychain.
- Use TLS for every request.
- Do not disable App Transport Security.

- Do not commit `.env` files.
- Do not print secrets in logs.
- Do not persist raw conversation audio by default.
- Do not persist transcript history by default.
- Hard-code the allowed OpenAI model and output language server-side.
- Rate-limit the token endpoint.
- Apply request and response size limits.

---

20. MVP Non-Goals

Do not implement these before the core translation path works:

- Two-way English-to-Portuguese translation
- Speaker enrollment
- Speaker identification
- Background listening
- Lock-screen operation
- CallKit integration
- Conversation recording
- Cloud transcript history
- User accounts
- Multi-device synchronization
- Custom voice selection
- Automatic dialect fine-tuning
- Complex audio enhancement
- Raw PCM WebSocket pipeline
- A general AI assistant or chat interface

---

21. Precision-Capture Fallback

Do not begin with this implementation.

If actual tests show that native WebRTC capture materially misses quiet speech, create a second transport:

```
AVAudioEngine
→ mono PCM16 resampling at 24 kHz
→ secure backend WebSocket relay
→ OpenAI Realtime Translation WebSocket
→ English transcript events
→ queued PCM output
```

This fallback provides direct access to microphone samples and permits custom gain, filtering, metering, and recording. Its costs are:

- More code
- More bandwidth
- Manual resampling
- Manual PCM playback
- More buffering
- Additional server infrastructure
- More potential for audio underruns and latency

OpenAI's WebSocket translation path expects base64-encoded, little-endian, 24 kHz PCM16 and returns output audio in PCM16 deltas.

Only build this path after a controlled A/B test proves it improves recognition enough to justify the complexity.

---

22. Implementation Phases

Phase 0: Repository and dependency setup

- Inspect the existing repository.
- Create or configure the SwiftUI target.
- Set minimum iOS version to iOS 17 unless the existing project requires another version.
- Add and pin the WebRTC dependency.
- Add the TypeScript token service.
- Add `.env.example`.
- Confirm both projects build.

Phase 1: Translation connectivity spike

Build the smallest possible test screen:

```
Connect
Disconnect
Connection state
Raw English transcript
```

Prove:

- Client-secret creation
- Native SDP exchange
- Microphone upload
- Remote audio reception
- Data-channel events

- English transcript streaming

Do not proceed until this works on a physical iPhone.

Phase 2: Production state and captions

Implement:

- `RealtimeTranslationClient`
- State machine
- Transcript assembler
- Large live captions
- In-memory history
- Automatic startup
- Pause and Stop
- Reconnection

Phase 3: Output modes

Implement:

- Automatic
- Listen
- Read
- Route monitoring
- Immediate mute on headphone disconnection
- Persisted selected mode

Phase 4: Audio quality

Implement:

- Preferred built-in microphone
- Effective-route verification
- Far-field/near-field session updates
- Video Chat/Voice Chat diagnostics
- Input-route display
- Microphone data-source inspection
- Interruption recovery

Phase 5: Evaluation support

Implement:

- Diagnostics screen
- Timing measurements
- Optional explicit local test recording
- Exportable redacted diagnostic report

- Profile comparison

Phase 6: Optional caption correction

Only after the direct live path performs acceptably:

- Glossary
- Caption-segment correction
- Corrected-caption indicator
- No delay to initial captions

---

23. Acceptance Criteria

The MVP is complete when all of the following pass on a physical iPhone.

Startup

- On first launch, the app requests microphone permission.
- On later launches, the app begins connecting without a Start button.
- The app reaches Listening without user prompting.
- A light haptic confirms listening has started.

Translation

- Spoken Portuguese produces incremental English captions.
- Translated English audio is received.
- Source Portuguese transcript can be enabled in Settings.
- Transcript deltas are concatenated without inserted spacing errors.
- The app does not issue `response.create`.

Output modes

- Automatic plus no private output produces captions with no audible English.
- Automatic plus private output produces captions and English audio.
- Read always mutes English audio.
- Listen enables English audio.
- Disconnecting headphones in Automatic mutes playback immediately.
- Changing modes does not recreate the API session.

Audio capture

- The app attempts to use the built-in iPhone microphone.
- The effective input route is displayed in Diagnostics.
- Audio is streamed continuously without local VAD gating.
- Built-in microphone defaults to far-field API noise reduction.
- Headset microphone defaults to near-field API noise reduction.
- Audio interruptions recover without requiring an app relaunch.

Reliability

- Temporary network loss triggers bounded reconnection.
- A new client secret is fetched for a new session.
- Existing captions remain visible during reconnection.
- The application does not crash when routes change.
- The application does not crash if the token server or OpenAI returns malformed JSON.

Privacy and security

- No permanent OpenAI key exists in the app bundle.
- Secrets are not logged.
- Audio and transcript history are not persisted by default.
- Diagnostic recording requires explicit action.
- Stopping the session clears in-memory conversation data.

Accessibility

- Captions support Dynamic Type.
- The interface remains usable at large accessibility sizes.
- Controls have VoiceOver labels.
- Connection state is not communicated by colour alone.
- Tap targets meet standard iOS sizing.

---

24. Required Deliverables

Codex should produce:

1. A compiling SwiftUI iOS project.
2. A working physical-device Realtime Translation connection.
3. A TypeScript token service.
4. Unit tests for:
5. Event decoding
6. Transcript concatenation
7. Caption segmentation
8. Output-mode routing
9. Connection-state transitions
10. A README containing:
11. Requirements
12. WebRTC dependency setup
13. Token-service setup

14. Environment variables
15. Xcode signing instructions
16. Physical-device run instructions
17. Troubleshooting
18. An `.env.example`.
19. No committed secrets.
20. A short architecture document.
21. A diagnostics checklist for comparing microphone profiles.

Keep the project compiling after every implementation phase.

Do not substitute mocked translation data for the real OpenAI connection in the completed MVP.

---

25. Final Engineering Priorities

Apply these priorities in order:

1. Preserve and capture quiet speech.
2. Produce accurate English meaning.
3. Avoid clipping the start of phrases.
4. Make captions available even when audio playback is muted.
5. Prevent translated audio from echoing into the conversation.
6. Start automatically with minimal interaction.
7. Recover cleanly from route and network changes.
8. Keep permanent credentials off the device.
9. Optimize latency.
10. Add advanced correction and speaker filtering later.

The initial application should remain deliberately narrow: open it, listen to Portuguese, and immediately understand the conversation in English.